Record - Week 03

Topic: Node.js and Express.js

Name: K. Sushanth Sai Krishna Roll No: A24126552152 Date: _____

Aim

To build a REST API server for expenses using Node.js and Express with routing and middleware.

Software Requirements

Node.js 18+, npm, VS Code, Postman

Procedure

1 Run npm init and install express.

2 Create server.js with JSON parsing and a logger middleware.

3 Create routes/api.js with GET and POST routes for expenses.

4 Mount the router at /api.

5 Start the server and test the endpoints in Postman.

Program

server.js

```
const express = require("express");
const apiRoutes = require("./routes/api");

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());

// simple logging middleware
app.use((req, res, next) => {
  console.log(`${req.method} ${req.url}`);
  next();
});

app.get("/", (req, res) => res.send("ExpenseWise API is running"));
app.use("/api", apiRoutes);

app.listen(PORT, () => console.log(`Server listening on port ${PORT}`));
```

routes/api.js

```
const express = require("express");
const router = express.Router();

let expenses = [
  { id: 1, title: "Groceries", amount: 850, category: "Food" },
  { id: 2, title: "Metro card recharge", amount: 500, category: "Travel" },
];
let nextId = 3;

router.get("/expenses", (req, res) => res.json(expenses));

router.get("/expenses/:id", (req, res) => {
  const e = expenses.find((x) => x.id === Number(req.params.id));
  if (!e) return res.status(404).json({ error: "Expense not found" });
  res.json(e);
});

router.post("/expenses", (req, res) => {
  const { title, amount, category } = req.body;
  if (!title || !(Number(amount) > 0) || !category) {
    return res.status(400).json({ error: "title, category and a positive amount required" });
  }
  const expense = { id: nextId++, title, amount: Number(amount), category };
  expenses.push(expense);
  res.status(201).json(expense);
});

module.exports = router;
```

package.json

```
{
  "name": "week-03-node-and-express",
  "version": "1.0.0",
  "description": "Express server with REST routes for expenses",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "node --watch server.js"
  },
  "dependencies": {
    "express": "^4.19.2"
  }
}
```

Output

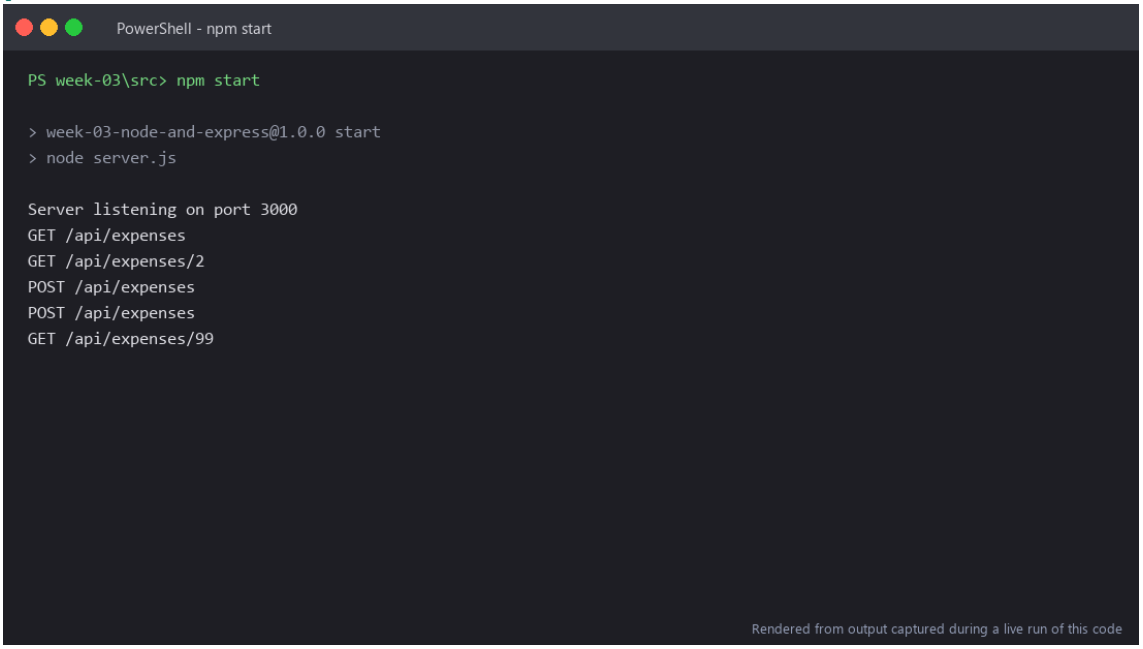


Fig: express server terminal



Fig: postman api response

Result

The program was executed successfully and the expected output was obtained.